

Homework 6: Directed Graph Theory Concepts & Definitions

Instructions:

- When you have completed these programs, UPLOAD THEM TO THIS ASSIGNMENT.
- A complete submission will include the files below, which lastname should be replaced with your last name.
 - HW6P1_directed_adjacency_matrix_lastname.m
 - HW6P2_cost_matrix_lastname.m
 - HW6P3_out_degrees_lastname.m
 - HW6P4_in_degrees_lastname.m
 - HW6P5_construct_directed_graph_lastname.m
 - HW6P6_get_components_lastname.m
 - HW6P7_min_cost_paths_lastname.m

Notes:

- Each of these files should contain a single function. So, there should be NO commands above or below the defined function inside the file. However, comments outside the function are ok.
- If you do not use descriptive variable names, you must describe every variable you define with code comments.
- The file names must match the format specified below to grade your programs. Misspellings and deviations from the naming convention will lead to errors when I run your code, and I will not manually retype your function names to make them work. So please double-check each file name before uploading it.

Programming Problems

Do not use any built-in MATLAB commands unless specified in the directions of the problem. Here is a list of approved MATLAB built-in functions you are free to use:

Matrix Size	length, numel, size, height, width
Matrix Operations	sum, prod, mod, floor, ceil, max, min, transpose
Matrix Generators	zeros, ones, inf, nan, primes
Type Detection	isempty, isequal, isnan, isfield, isinf

Function: `HW6P1_directed_adjacency_matrix_lastname.m`

Description: This function constructs the adjacency matrix for a directed graph represented by the structure `G`.

The adjacency matrix is a square matrix where each element represents the number of directed edges from vertex to vertex

Inputs: `G` (struct): A structure representing the graph containing, at minimum, the following fields:

- `G.n` (integer): The number of vertices in the graph.
- `G.m` (integer): The number of edges in the graph.
- `G.edges` (m x 2): A matrix where each row represents an edge. connecting two vertices.

Outputs: `A` (n x n): The adjacency matrix, where n is the number of vertices in the graph.

Ensure your function handles cases with no edges (empty edge list).

Comments: Consider edge cases where multiple edges between two vertices may be present.

Test Cases:

Command Window

Expected Output

```
G.vertices = 1:3;  
G.n = 3;  
G.m = 2;  
G.edges = [1, 2; 2, 3];  
A = HW6P1_directed_adjacency_matrix(G);
```

```
A =  
    0    1    0  
    0    0    1  
    0    0    0
```

```
G.vertices = 1:2;  
G.n = 2;  
G.m = 2;  
G.edges = [1, 1, 4; 1, 2, 5];  
A = HW6P1_directed_adjacency_matrix(G);
```

```
A =  
    1    1  
    0    0
```

```
G.vertices = 1:4;  
G.n = 4;  
G.m = 5;  
G.edges = [3 4; 4 4; 1 4; 4 3; 1 1];  
A = HW6P1_directed_adjacency_matrix(G);
```

```
A =  
    1    0    0    1  
    0    0    0    0  
    0    0    0    1  
    0    0    1    1
```

Function: HW6P2_cost_matrix_lastname.m

Description: This function constructs the cost matrix for a directed graph represented by the structure **G**. The cost matrix is a square matrix where each element represents the cost of the edge from vertex to vertex . If no edge exists, the cost is set to **inf**.

Inputs: **G** (struct): A structure representing the graph containing, at minimum, the following fields:

- G.n** (integer): The number of vertices in the graph.
- G.m** (integer): The number of edges in the graph.
- G.edges** (m x 3): A matrix where each row represents an edge connecting two vertices with a cost, formatted as [from, to, cost].

Outputs: **C** (n x n): The cost matrix, where is the number of vertices in the graph. Nonexistent edges have a cost of **inf**.

Comments: Ensure your function handles cases with no edges (empty edge list).

Handle self-loops with their specified cost; otherwise, give them a zero cost.

Verify that multiple edges between the same vertices result in the minimum cost being used.

Test Cases:

Command Window

Expected Output

```
G.n = 3;  
G.m = 2;  
G.edges = [1, 2, 5; 2, 3, 10];  
C = HW6P2_cost_matrix(G);
```

```
C =  
    0     5   inf  
   inf     0   10  
   inf   inf     0
```

```
G.n = 2;  
G.m = 2;  
G.edges = [1, 1, 3; 1, 2, 7];  
C = HW6P2_cost_matrix(G);
```

```
C =  
    3     7  
   inf     0
```

```
G.n = 4;  
G.m = 5;  
G.edges = [1, 2, 6; 1, 3, 3; 2, 4, 8; 3, 4, 2; 4, 1, 7];  
C = HW6P2_cost_matrix(G);
```

```
C =  
    0     6     3   inf  
   inf     0   inf     8  
   inf   inf     0     2  
    7   inf   inf     0
```

Function: HW6P3_out_degrees_lastname.m

Description: This function calculates the out-degrees of all vertices in a directed graph represented by the structure **G**. The out-degree of a vertex is the number of edges leaving the vertex.

Inputs: `G` (struct): A structure representing the graph containing, at minimum, the following fields:

- `G.n` (integer): The number of vertices in the graph.
- `G.m` (integer): The number of edges in the graph.
- `G.edges` (m x 2): A matrix where each row represents an edge [from, to].

Outputs: `out_degrees` (1 x n): A row vector where each element `out_degrees(i)` represents the number of edges leaving vertex `i`.

Comments: Ensure your function handles cases with no edges (empty edge list).

Support graphs with self-loops, where an edge starts and ends at the same vertex.

Test Cases:

Command Window

Expected Output

```
G.n = 3;  
G.m = 2;  
G.edges = [1, 2; 2, 3];  
out_degrees = HW6P3_out_degrees(G);
```

```
out_degrees =  
    1    1    0
```

```
G.n = 2;  
G.m = 2;  
G.edges = [1, 1; 1, 2];  
out_degrees = HW6P3_out_degrees(G);
```

```
out_degrees =  
    2    0
```

```
G.n = 4;  
G.m = 5;  
G.edges = [3, 4; 4, 4; 1, 4; 4, 3; 1, 1];  
out_degrees = HW6P3_out_degrees(G);
```

```
out_degrees =  
    2    0    1    2
```

Function: `HW6P4_in_degrees_lastname.m`

Description: This function calculates the in-degrees of all vertices in a directed graph represented by the structure `G`. The in-degree of a vertex is the number of edges entering the vertex.

Inputs: `G` (struct): A structure representing the graph containing, at minimum, the following fields:

- `G.n` (integer): The number of vertices in the graph.
- `G.m` (integer): The number of edges in the graph.
- `G.edges` (m x 2): A matrix where each row represents an edge [from, to].

Outputs: `in_degrees` (1 x n): A row vector where each element `in_degrees(i)` represents the number of edges entering vertex `i`.

Comments: Ensure your function handles cases with no edges (empty edge list).

Support graphs with self-loops, where an edge starts and ends at the same vertex.

Test Cases:

Command Window

Expected Output

```
G.n = 3;  
G.m = 2;  
G.edges = [1, 2; 2, 3];  
in_degrees = HW6P4_in_degrees(G);
```

```
in_degrees =  
    0    1    1
```

```
G.n = 2;  
G.m = 2;  
G.edges = [1, 1; 1, 2];  
in_degrees = HW6P4_in_degrees(G);
```

```
in_degrees =  
    1    1
```

```
G.n = 4;  
G.m = 5;  
G.edges = [3, 4; 4, 4; 1, 4; 4, 3; 1, 1];  
in_degrees = HW6P4_in_degrees(G);
```

```
in_degrees =  
    1    0    1    3
```

Function: `HW6P5_construct_directed_graph_lastname.m`

Description: This function constructs a directed graph structure from a given set of vertices and edges. It also computes the graph properties listed in the output details below.

Inputs: `vertices` (n x 1, integer): A list of vertex IDs.

`edges` (m x 2 or m x 3, integer): A list of edges where each row contains the starting and ending vertex IDs.

`graph_name` (string): A name for the graph stored as part of the output structure.

Outputs: `G` (struct): A structure representing the graph containing the following fields:

1. `G.name` (string): The name of the graph as provided in `graph_name`.
2. `G.vertices` (n x 1, integers): List of vertex index IDs
3. `G.edges` (m x 3): A matrix where each row represents an edge connecting two vertices.
4. `G.n` (integer): The number of vertices in the graph.
5. `G.m` (integer): The number of edges in the graph.
6. `G.adj_mat` (n x n double array): The adjacency matrix of the graph.
7. `G.cost_mat` (n x n double array): The cost/weight matrix of the graph.
8. `G.out_degrees` (n x 1, integers): List of vertex out-degrees.

9. `G.in_degrees` (n x 1, integers): List of vertex in-degrees.

Note: The values of the fields 6 through 9 should come from the output of `HW6P1` - `HW6P4`, respectively.

Comments: If the edges matrix is input without weights (i.e., a 3rd column), give each edge a default weight of 1.

Function: `HW6P6_get_components_lastname.m`

Description: This function identifies all connected components in a directed graph represented by the structure `G`. A component is a subgraph where any two vertices are connected by a path.

Inputs: `G` (struct): A structure representing the graph containing, at minimum, the following fields:

- `G.n` (integer): The number of vertices in the graph.
- `G.edges` (m x 2): A matrix where each row represents an undirected edge [vertex1, vertex2].

Outputs: `components` (1 x k): A cell array where each element `components{i}` is a vector containing the vertices in the `i`th connected component.

Comments: Ensure your function handles cases with no edges (empty edge list).

Support graphs with self-loops, where an edge starts and ends at the same vertex.

In MATLAB, cell arrays are like matrices, except you can store different types of objects. This new structure is needed to store each path as a vector.

Test Cases:

Command Window

Expected Output

```
V = 1:5;
E = [ 1 2;
      2 3;
      4 5 ];
G = HW6P5_construct_directed_graph(V,E,'G');
components = HW6P6_get_components(G);
```

```
components =
    {[1 2 3], [4 5]}
```

```
V = 1:3;
E = [ 1 2 5;
      2 3 1;
      1 3 7 ];
G = HW6P5_construct_directed_graph(V,E,'G');
components = HW6P6_get_components(G);
```

```
components =
    {[1 2 3]}
```

```
V = 1:4;
E = [];
G = HW6P5_construct_directed_graph(V,E,'G');
components = HW6P6_get_components(G);
```

```
components =
    {[1], [2], [3], [4]}
```

Function: `HW6P7_min_cost_paths_lastname.m`

Description: Given a starting vertex, this function computes both the shortest distance and path to every other vertex in a directed graph `G`.

Inputs: `G` (struct): A structure representing the directed graph.

`source` (integer): The index of the source vertex from which shortest paths are calculated.

Outputs: `distances` (1 x n): A row vector where each element `distances(i)` represents the shortest distance from the source vertex to vertex `i`. If a vertex is unreachable, its distance is `inf`.

`paths` (1 x n): A cell array where each element `paths{i}` is a vector representing the shortest path from the source vertex to vertex `i`.

Comments: Ensure your function handles cases with no edges (empty edge list).

In MATLAB, cell arrays are like matrices, except you can store different types of objects. This new structure is needed to store each path as a vector.

Test Cases:

Command Window

Expected Output

```
V = 1:6;
E = [ 1 2 1;
      1 3 7;
      1 4 6;
      2 3 2;
      3 4 8;
      3 5 3;
      4 5 4;
      4 6 6;
      5 6 4 ];
G = HW6P5_construct_directed_graph(V,E,'G');
[distances, paths] = HW6P7_min_cost_paths(G,1)
```

```
distances =
    0     1     3     6     6    10
paths =
    {[1], [1 2], [1 2 3], [1 4], ...
     [1 2 3 5], [1 2 3 5 6]}
```

```
V = 1:4;
E = [ 1 2 3;
      1 3 1;
      2 4 2;
      3 4 6 ];
```

```
distances =
    0     3     1     5
paths =
    {[1], [1 2], [1 3], [1 2 4]}
```

```
G = HW6P5_construct_directed_graph(V,E,'G');  
[distances, paths] = HW6P7_min_cost_paths(G,1)
```

```
V = 1:3;  
E = [ 1 2 5;  
      2 3 4 ];  
G = HW6P5_construct_directed_graph(V,E,'G');  
[distances, paths] = HW6P7_min_cost_paths(G,2);
```

```
distances =  
    Inf     0     4  
paths =  
    {0×0 double, [2], [2 3]}
```

```
V = 1:3;  
E = [];  
G = HW6P5_construct_directed_graph(V,E,'G');  
[distances, paths] = HW6P7_min_cost_paths(G,3);
```

```
distances =  
    Inf    Inf     0  
paths =  
    {0×0 double, 0×0 double, [3]}
```